

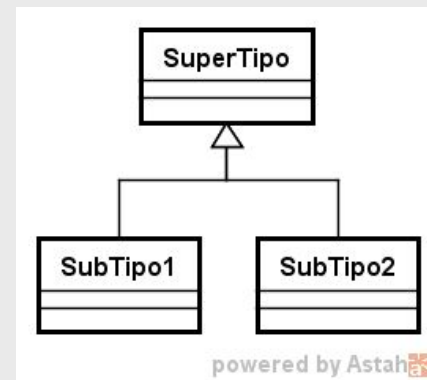


Classes Abstratas e Herança em C++

Prof. Dr. Valter Vieira de Camargo

POO - 2026/1

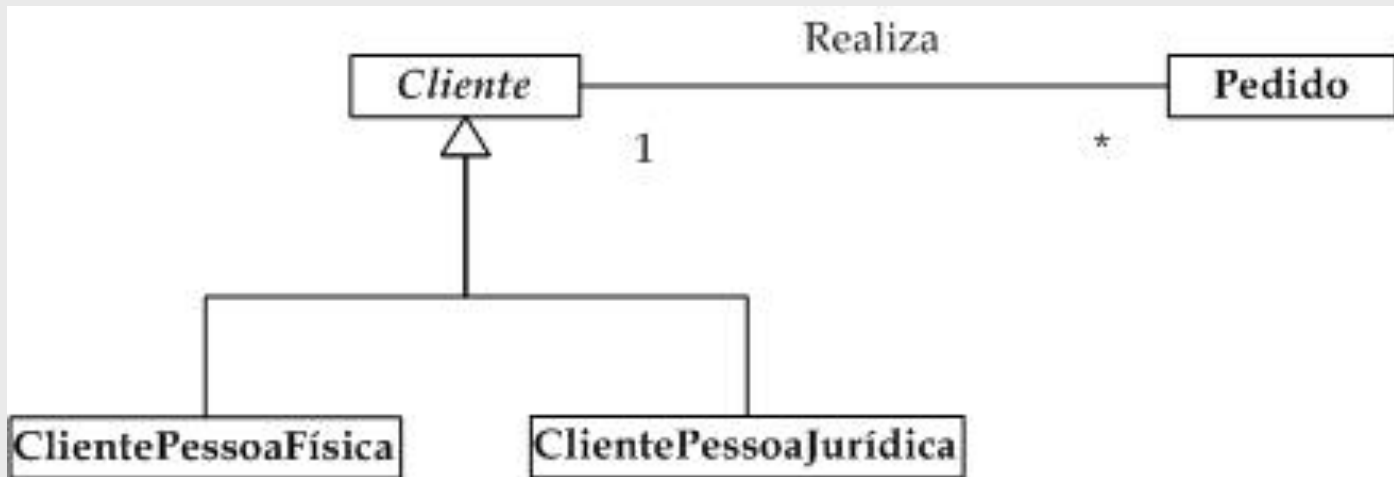
Herança/Generalização



- ◆ Usado para representar hierarquia de tipos e subtipos
- ◆ SuperTipo geralmente é uma classe abstrata, uma abstração para as classes que estão embaixo (**um Contrato**)
- ◆ Normalmente, identifica-se um relacionamento de herança fazendo-se a pergunta “É um” ?
 - Se a resposta for “sim”, possivelmente é um relacionamento de herança
 - Cliente físico é um Cliente ?
 - Carro é um Veículo ?
 - Vendedor comissionado é um Vendedor ?
 - Leão é um animal ?
- ◆ Herança significa que subclasses herdam as características (atributos, relacionamentos e métodos) de sua superclasse

Herança/Generalização

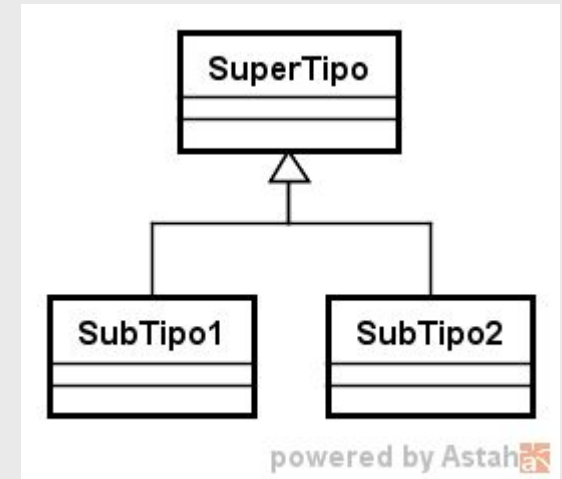
- Não somente atributos e operações, mas também associações são herdadas pelas subclasses.
- No exemplo abaixo, por herança, cada subclasse também está associada a Pedido, já que o relacionamento nada mais é do que um atributo do tipo Pedido dentro da classe Cliente.



Herança/Generalização

Terminologia

- superclasse X subclasse
- supertipo X subtipo
- classe base X classe herdeira
- classe de generalização X classe de especialização
- ancestral e descendente (herança em vários níveis)



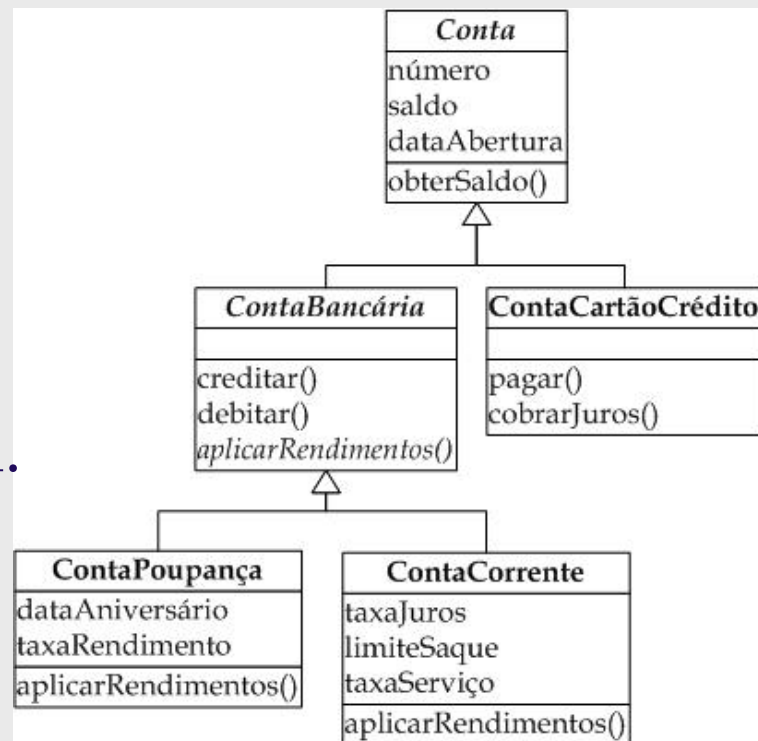
Herança/Generalização

- Transitividade:

- uma classe em uma hierarquia herda propriedades e relacionamentos de todos os seus ancestrais.
- Ou seja, a herança pode ser aplicada em vários níveis, dando origem a *hierarquia de generalização*.
- Uma classe que herda propriedades de uma outra classe pode ela própria servir como superclasse.

- Assimetria

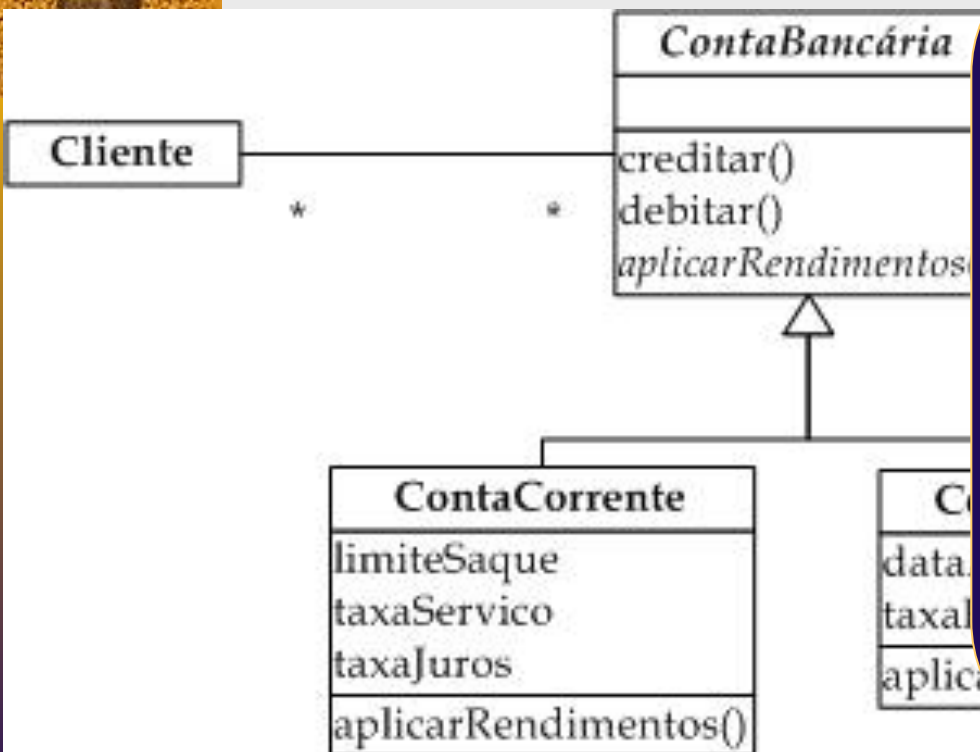
- dadas duas classes A e B, se A for uma generalização de B, então B não pode ser uma generalização de A.
 - Não pode haver ciclos na hierarquia.



Herança/Generalização

Conceito Importante !

- Todo objeto da classe Filha **É** também do mesmo tipo da classe Pai
- O tipo do objeto da classe Filha é também do mesmo tipo da classe Pai



Um objeto do tipo **ContaCorrente** é do tipo **ContaCorrente** mas também é do tipo **ContaBancária** (entenda bem isso)

Isso significa que um objeto do tipo **ContaCorrente** pode ser atribuído a um objeto do tipo **ContaBancária**

```
ContaCorrente minhaCC;  
ContaBancaria* conta = &minhaCC;
```


Herança/Generalização - Exemplos Possíveis

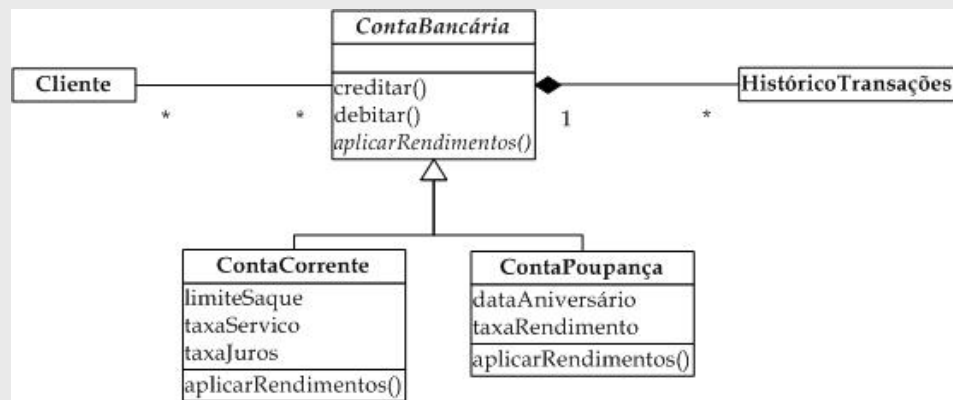


Classe Base	Classes Derivada
Estudante	Estudante_graduação Estudante_pós_graduação
Figura	Circulo Triângulo Retângulo
Empréstimo	Empréstimo_para_carro Empréstimo_para_casa_própria Empréstimo_pessoal
Empregado	Empregado_chão_de_fábrica Gerente Diretor
Conta	Conta_corrente Conta_poupança

Classes Abstratas

- São classes que não podem ter instâncias, não é possível criar objetos de classes abstratas, servem só como contrato, como modelo, como molde para as classes filhas
- Propriedades (atributos e métodos) comuns a diversas classes podem ser organizadas e definidas em uma classe abstrata, a partir da qual as primeiras herdam.
- Subclasses de uma classe abstrata também podem ser abstratas, mas a hierarquia deve terminar em uma ou mais classes concretas.
- Na UML elementos em **Itálico** são abstratos

Conselho: Quase sempre utilizem classes abstratas como classes-base de uma hierarquia. A exceção é usar classe concretas.





Classes abstratas em C++

- ♦ C++ não tem uma palavra chave “abstract” para classes
- ♦ Uma classe abstrata é aquela que possui pelo menos uma função “virtual pura”

```
class Forma {  
public:
```

```
    virtual void desenhar() = 0; // função virtual pura  
};
```

= 0 é o que faz com que essa função virtual seja pura

- ♦ A função virtual pura é “obrigatória” de ser implementada nas classes derivadas

Classes abstratas em C++

```
class Forma {  
public:  
    virtual void desenhar() = 0; // função virtual pura  
};
```

Se eu fizer no Main →

Forma forma (.....); ou

Forma* forma = new Forma (....)

main.cpp: In function 'int main()':

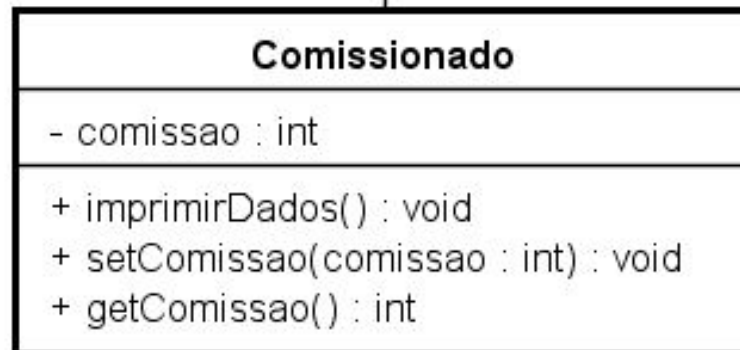
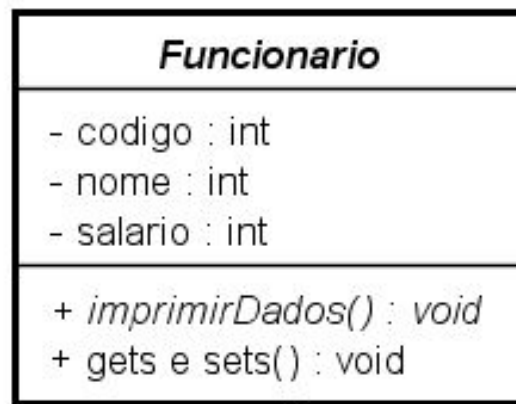
main.cpp:19:11: error: cannot declare variable 'forma' to be of abstract type 'Forma'

```
19 |     Forma forma;
```

```
    ^~~~~~
```



Exemplo



- Meu sistema só tem dois tipos de funcionários
- Tudo que for comum devo colocar na classe Funcionario
- Deixo embaixo só o que for específico
- Se um novo tipo de funcionário aparecer, basta criar uma nova classe só com o que for específico
- o método imprimirDados() possui a mesma assinatura, mas implementações diferentes

Classe Funcionario

```
#include <iostream>
#include <string>
using namespace std;
```

```
// Classe base abstrata
```

```
class Funcionario {
```

```
private:
```

```
    int codigo;
```

```
    string nome;
```

```
    double salario;
```

```
public:
```

```
    Funcionario(int codigo, const string& nome, double salario)
        : codigo(codigo), nome(nome), salario(salario) {}
```

```
...
```

```
    // Getters
```

```
    // Setters
```

```
    // Método virtual puro → torna a classe Funcionario abstrata
    virtual void imprimirDados() const = 0;
```

```
    // Virtual destructor: boa prática em classes com herança
    virtual ~Funcionario() {}
```

```
};
```



Classe Assalariado

É assim que se faz herança em C++

// Classe Assalariado herda publicamente de Funcionario

class Assalariado : public Funcionario {

public:

Assalariado(int c, const string& n, double s)

: Funcionario(c, n, s) {}

Cuidado com o construtor !

void imprimirDados() const override {

cout << "Funcionario Assalariado:" << endl;

cout << "Codigo: " << getCodigo() << endl;

cout << "Nome: " << getNome() << endl;

cout << "Salario: R\$ " << getSalario() << endl;

cout << "-----" << endl;

}

};

override

- diz ao compilador que você está sobrescrevendo esse método de propósito
- dá erro de compilação se você não seguir a assinatura exata
- sem o override, o compilador deixa passar, mas você corre o risco de estar criando outro método se a assinatura for diferente



Classe Assalariado

// Classe Assalariado herda publicamente de Funcionario

```
class Assalariado : public Funcionario {
```

```
public:
```

```
    Assalariado(int c, const string& n, double s)  
        : Funcionario(c, n, s) {}
```

```
void imprimirDados() const override {  
    cout << "Funcionario Assalariado:" << endl;  
    cout << "Codigo: " << getCodigo() << endl;  
    cout << "Nome: " << getNome() << endl;  
    cout << "Salario: R$ " << getSalario() << endl;  
    cout << "-----" << endl;
```

```
};
```

- Como os atributos do pai são privados, eles não são diretamente acessíveis na classe filha. Então, devem ser acessados com os métodos get.



Classe Comissionado

// Classe Comissionado herda publicamente de Funcionario

```
class Comissionado : public Funcionario {
```

private:

```
    double comissao;
```

← tem um atributo adicional

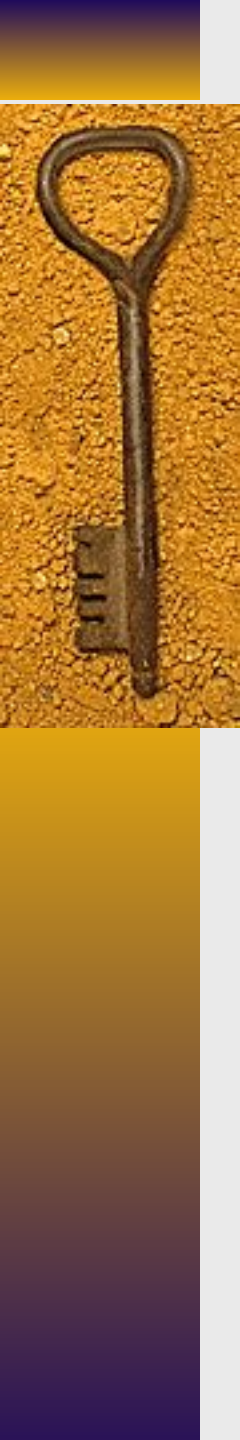
public:

```
    Comissionado(int c, const string& n, double s, double com)
        : Funcionario(c, n, s), comissao(com) {}
```

// Get e set do atributo “comissão” vão aqui ...

```
void imprimirDados() const override {
    cout << "Funcionario Comissionado:" << endl;
    cout << "Codigo: " << getCodigo() << endl;
    cout << "Nome: " << getNome() << endl;
    cout << "Salario base: R$ " << getSalario() << endl;
    cout << "Comissao: R$ " << comissao << endl;
    cout << "Salario total: R$ " << (getSalario() + comissao) << endl;
    cout << "-----" << endl;
}
};

};
```



Sobre o “override”

- ♦ é usado para indicar que um método está sobreescrevendo um método virtual da classe base;
- ♦ se você errar alguma coisa no nome do método na classe derivada, o compilador detecta;
- ♦ só pode usar override se o método base for virtual
- ♦ usar override é uma boa prática

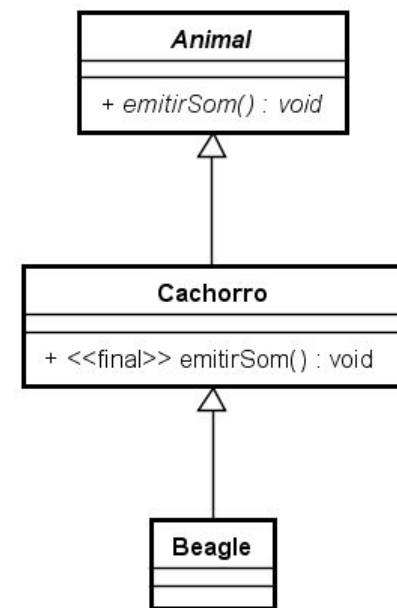
Sobre o “final”

- ♦ Em hierarquia de herança, serve para impedir herança ou sobrescrita de métodos ou classes;
- ♦ em métodos
 - ao marcar um método virtual como final, está dizendo que nenhuma classe derivada poderá sobrescrevê-lo novamente.

```
class Animal {  
public:  
    virtual void emitirSom() const {  
        cout << "Som genérico\n";  
    }  
};
```

```
class Cachorro : public Animal {  
public:  
    void emitirSom() const final { // ninguém poderá  
        sobrescrever isso depois  
        cout << "Au au!\n";  
    }  
};
```

```
class Beagle : public Cachorro {  
public:  
    void emitirSom() const override { // ERRO: não pode  
        sobrescrever, pois é final  
        cout << "Beagle late!\n";  
    }  
};
```



powered by Astah

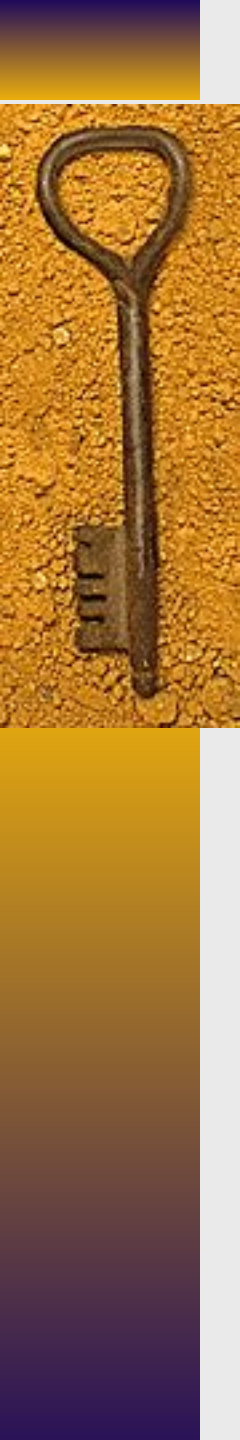


Sobre o “final”

- ♦ Se uma classe é marcada como “final”, ela não pode ser mais herdada

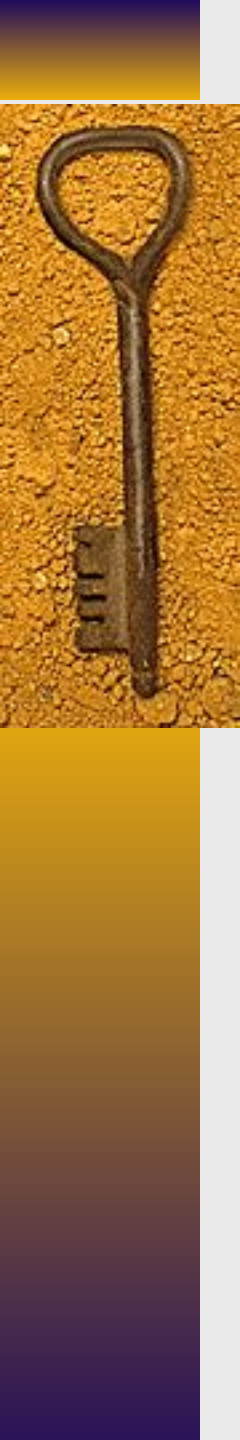
```
class Veiculo final {  
    // ...  
};
```

```
// ERRO: não pode herdar de Veiculo  
class Carro : public Veiculo {  
    // ...  
};
```



Porque usar “final”

- ◆ Segurança → você tem certeza que determinado comportamento não poderá ser alterado por subclasses
- ◆ Otimização → o compilador pode otimizar melhor chamadas para métodos “final”
- ◆ Design mais claro → deixa evidente onde a hierarquia termina



Destrutores

- ♦ Destrutor é um método especial que é automaticamente chamado quando um objeto é destruído
 - destruído quando sai do escopo (automaticamente) ou
 - quando é explicitamente chamado (por meio de `delete`)
- ♦ Tem o objetivo de liberar memória alocada pelo objeto durante sua vida útil
- ♦ Características:
 - Nome do destrutor é sempre o nome da classe precedido por um `~`
 - Não tem tipo de retorno
 - Não pode receber parâmetros
- ♦ Ele é chamado automaticamente ou quando usa-se o “delete”

Destrutores

- ◆ Problema se não declarar o destrutor

```
class Carro {
```

```
private:
```

```
    int* rodas;
```

```
public:
```

```
    Carro() {
```

```
        rodas = new int(4); // memória alocada dinamicamente
```

```
    }
```

```
    // Sem destrutor definido!
```

```
};
```

```
int main() {
```

```
    Carro* c = new Carro();
```

```
    delete c; // Chamará o destrutor padrão
```

```
}
```

Como você não declarou um destrutor, o compilador gera um destrutor padrão, que não sabe que rodas precisa de delete. Resultado: memória alocada com new não será liberada — isso é um vazamento de memória.



Destruutores

- ◆ Problema se não declarar o destrutor

```
class Carro {
```

```
private:
```

```
    int* rodas;
```

```
public:
```

```
    Carro() {
```

```
        rodas = new int(4); // memória alocada dinamicamente
```

```
    }
```

```
    ~Carro() {
```

```
        delete rodas; // Liberação de memória
```

```
    }
```

```
};
```

Obs. Se a classe possui somente atributos de tipos primitivos (string, int, vector, map,...), podemos confiar no destrutor “default”, que é automaticamente criado pelo compilador

Isto é o correto !



Destrutores em Hierarquias

- ♦ Regra de ouro → sempre declarar o destrutor da classe base como virtual

```
class Base {  
public:  
    ~Base() { // não é virtual .... vai dar problema ...  
        std::cout << "Destruindo Base\n";  
    }  
};  
  
class Derivada : public Base {  
public:  
    ~Derivada() {  
        std::cout << "Destruindo Derivada\n";  
    }  
};
```

O ponteiro é do tipo Base, mas `~Base()` **não é virtual**, então **apenas o destrutor de Base é chamado**. O de Derivada é **ignorado** → isso pode deixar recursos sem liberar (ex: delete nunca chamado para dados da Derivada).

```
Base* obj = new Derivada();  
delete obj; // comportamento indefinido!
```



Destrutores em Hierarquias

- ♦ Regra de ouro → sempre declarar o destrutor da classe base como virtual

```
class Base {  
public:  
    ~Base() { // não é virtual .... vai dar problema ...  
        std::cout << "Destruindo Base\n";  
    }  
};  
  
class Derivada : public Base {  
public:  
    ~Derivada() {  
        std::cout << "Destruindo Derivada\n";  
    }  
};
```

Normalmente, basta declarar o destrutor virtual na classe base sem precisar declarar destrutores nas derivadas.

```
Base* obj = new Derivada();  
delete obj; // comportamento indefinido!
```

Quando declarar destrutor nas classes derivadas ?

- ♦ resposta → quando você aloca manualmente memória (new) dentro da classe filha
- ♦ Veja exemplo →



```
class Motor {
public:
    Motor() {
        std::cout << "Motor construído\n";
    }

    ~Motor() {
        std::cout << "Motor destruído\n";
    }
};
```

```
class Veiculo {
public:
    virtual ~Veiculo() {
        std::cout << "Destrutor de
Veiculo\n";
    }
};
```

```
class Carro : public Veiculo {
private:
    Motor* motor; // ponteiro

public:
    Carro() {
        motor = new Motor(); // alocação dinâmica
        std::cout << "Carro construído\n";
    }
};
```

```
int main() {
    Veiculo* v = new Carro();
    delete v;
    return 0;
}
```

// Necessário declarar o destrutor!

```
~Carro() {
    delete motor; // liberar a memória manualmente
    std::cout << "Destrutor de Carro\n";
}
```


Formas de instanciar os objetos

- Como já foi mostrado em aula há duas formas de instanciar objetos
 - Instanciação automática (no Stack)
 - Você simplesmente **declara** o objeto como uma variável normal.
 - A memória dele é alocada na **stack** (pilha de execução).
 - A destruição do objeto é **automática** quando ele sai do escopo (por exemplo, ao terminar o **main** ou uma função). O compilador chama automaticamente os destrutores.
 - Instanciação dinâmica (no Heap)
 - Você cria o objeto usando a palavra-chave **new**.
 - A memória é alocada no **heap** (área de memória gerenciada manualmente).
 - **Você é responsável** por liberar essa memória usando **delete** depois.



Instanciação Estática - Exemplo 1



```
int main() {  
    Assalariado a(1, "Maria", 3000.0);  
    Comissionado c(2, "João", 2000.0, 500.0);  
  
    a.imprimirDados();  
  
    c.imprimirdados();  
  
    return 0;  
}
```

Instanciação Estática - Exemplo 2



```
int main() {  
    Assalariado a(1, "Maria", 3000.0);  
    Comissionado c(2, "João", 2000.0, 500.0);
```

```
    Funcionario* f1 = &a;  
    Funcionario* f2 = &c;
```

```
    f1->imprimirDados();
```

```
    cout << endl;
```

```
    f2->imprimirDados();
```

```
    return 0;
```

```
}
```

Neste caso estamos começando a usar Polimorfismo e é o jeito correto de usar objetos de classes filhas de uma hierarquia.



Instanciação Dinâmica - Exemplo

```
int main() {
```

```
    Funcionario* f1;
```

```
    Funcionario* f2;
```

```
    f1 = new Assalariado(1, "Maria", 3000.0);
```

```
    f2 = new Comissionado (2, "João", 2000.0, 500.0);
```

```
    f1->imprimirDados();
```

```
    f2->imprimirDados();
```

```
    delete f1;
```

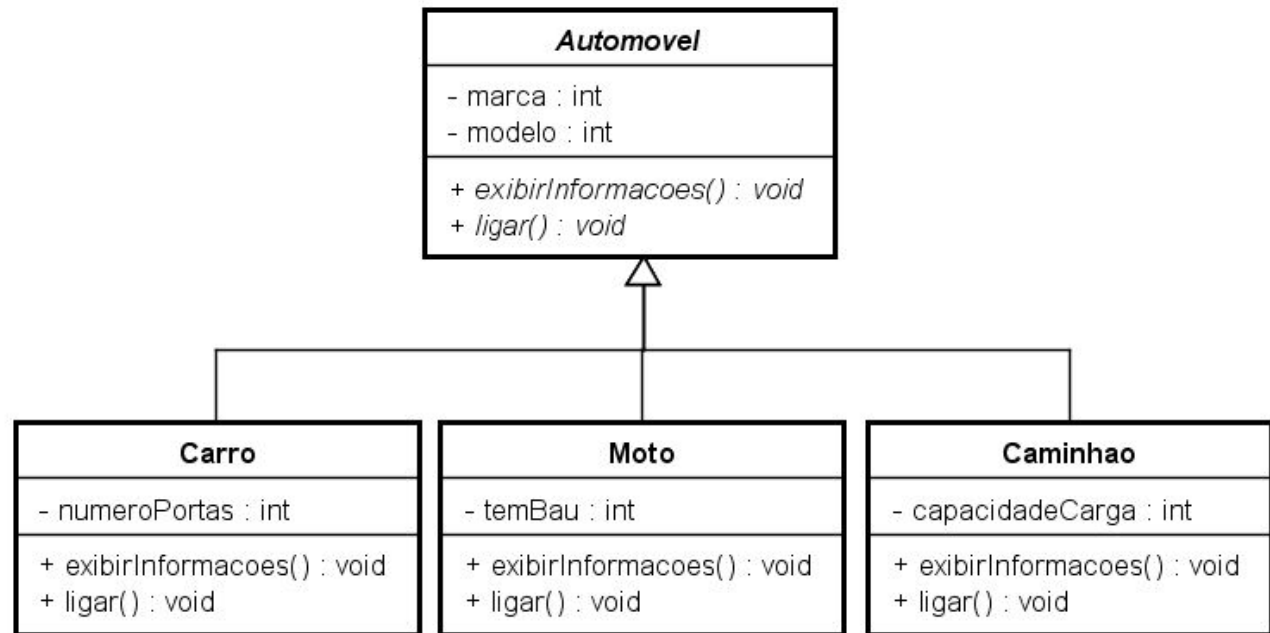
```
    delete f2;
```

```
    return 0;
```

```
}
```

Exercício

Implemente a hierarquia abaixo

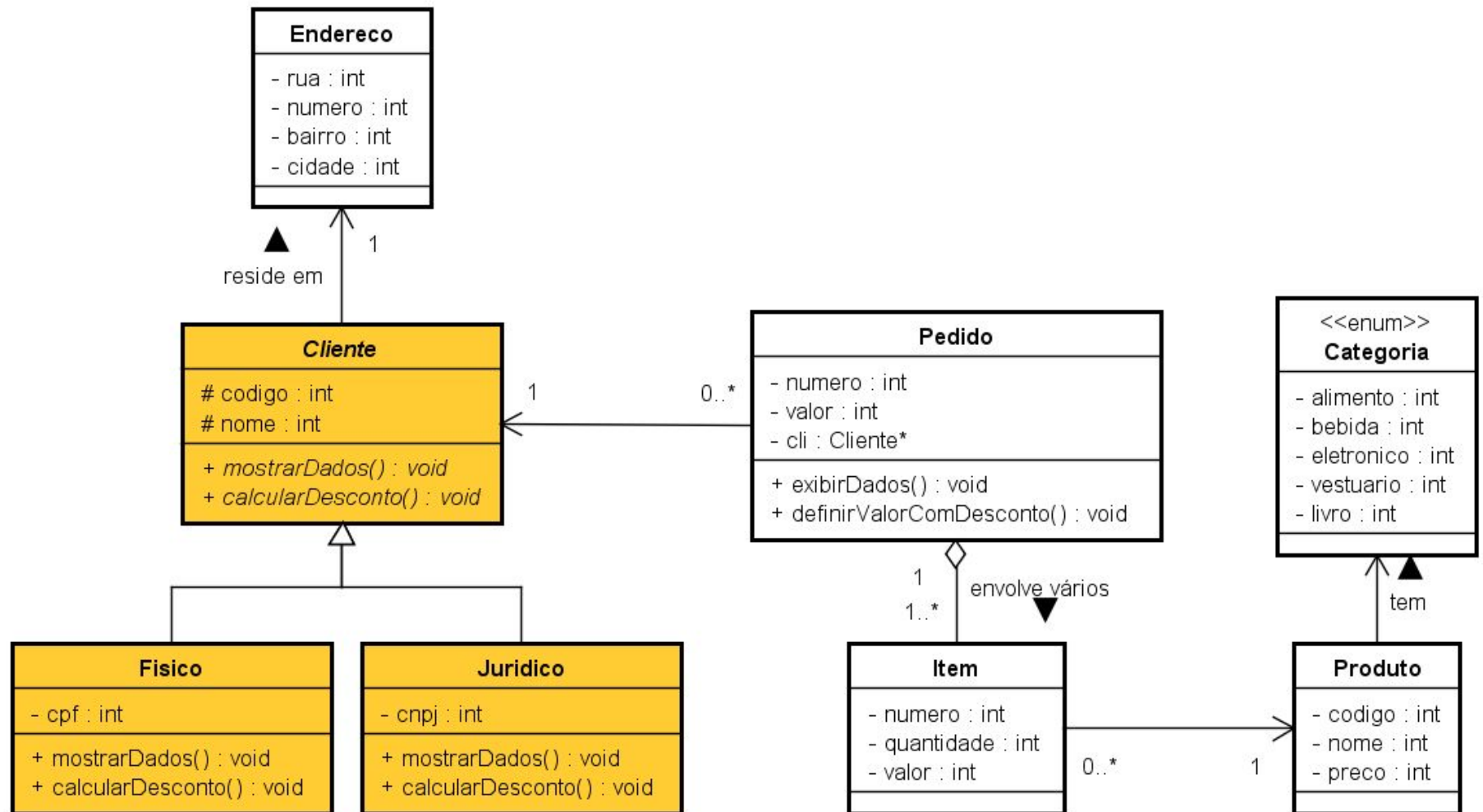


powered by Astah

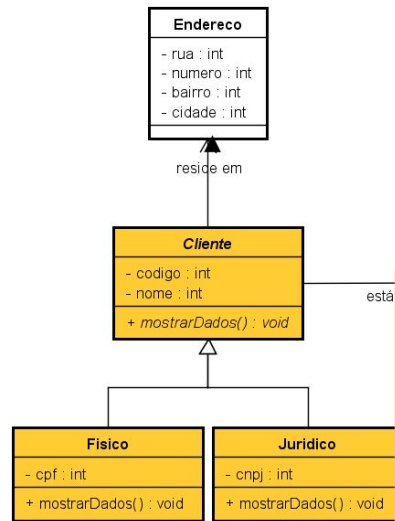
No Main

- Faça instâncias dinâmicas e automáticas dos objetos
- Use os métodos abstratos (invoque-os)

Evolução do sistema de pedidos



Evolução do sistema de pedidos



- Criar a hierarquia para Cliente
- Só existem dois tipos de clientes no sistema
- Implementar o método abstrato mostrarDados()
- Modificar o menu para que seja possível cadastrar os dois tipos de clientes